

- Lập trình vi điều khiển ta có thể thực hiện bằng sơ đồ giải thuật hoặc code
 - sơ đồ giải thuật : thông qua các khối : gán , ra quyết định , lặp , hàm con . giúp ta có bản thiết kế chương trình
 - code : dùng một ngôn ngữ lập trình (C++) để biên dịch các sơ đồ giải thuật
- Sự liên kết giữa sơ đồ giải thuật và code như sau

sơ đồ giải thuật	cấu trúc code
Khối gán	khai báo biến
Khối ra quyết định	câu lệnh if , switch...case
khối lặp	câu lệnh for , câu lệnh while
khối chương trình con	xây dựng hàm

Khai báo biến:

Cách khai báo biến : kiểu_dữ_liệu tên_biến = giá_trị

kiểu_dữ_liệu arduino	kích thước	ý nghĩa
char	1 byte	số nguyên từ -128 đến 127
unsigned char	1 byte	số nguyên dương 0 đến 255
int	2 byte	kiểu số nguyên : -32.768 đến 32.767
unsigned int	1 byte	kiểu số nguyên : 0 đến 65.535
long	4 byte	kiểu số nguyên: -2.147.483.648 đến 2.147.483.647
unsigned long	4 byte	kiểu số nguyên dương: 0 đến 4.294.967.295
float /double	2 byte	kiểu số thực (dấu phẩy động): nằm trong khoảng \$ \pm 16.777.216\$
bool	1 byte	đây là kiểu luận lý lưu đúng/sai giá trị true , false
char[]	tùy thuộc độ dài chuỗi	đây là kiểu chuỗi để lưu "hello" , "open" , "close"

- Kích thước kiểu dữ liệu sẽ thay đổi tùy vào kiến trúc dòng vi điều khiển .Ta hay dùng thư viện chuẩn hóa để chọn kiểu dữ liệu hơn với kiểu số nguyên

kiểu dữ_liệu số nguyên chuẩn hóa	kích thước	tương đương trong arduino
int8_t	1 byte	char
uint8_t	1 byte	unsigned char
int16_t	2 byte	int

kiểu dữ liệu số nguyên chuẩn hóa	kích thước	tương đương trong arduino
uint16_t	2 byte	unsigned int
int32_t	4 byte	long
uint32_t	4 byte	unsigned long

Chú ý lời khuyên

- Việc khởi tạo biến để lưu cũng giống như việc ta nhét dữ liệu vào ram vi điều khiển , ram vi điều khiển rất hạn chế , cho nên việc chọn kiểu thích hợp là yêu cầu bắt buộc
- Ví dụ lưu chân vi điều khiển : ta chỉ cần chọn kiểu 1byte là được, không nên chọn kiểu int
- Nên dùng kiểu dữ liệu chuẩn hóa cho số nguyên.

Ví dụ

- Khai báo đèn led chân 1: uint8_t led = 1
- Khai báo nhiệt độ : float nhiet_do = 7.5
- Khai báo lệnh điều khiển : char lenh_1[]="mo cua"

Vị trí đặt biến

- Biến đặt nằm ngoài tất cả các hàm : Biến toàn cục. Đặt biến toàn cục có thể truy xuất bất cứ đâu trong chương trình
- Biến đặt nằm trong hàm : biến cục bộ. Biến này chỉ tồn tại khi nằm trong phạm vi hàm , ra khỏi hàm biến này sẽ mất để dọn dẹp bộ nhớ

ví dụ: khi khai báo chân cho một led : uint8_t led = 1 thì biến led bắt buộc khai báo toàn cục để có thể gọi bất cứ đâu trong chương trình

Địa chỉ biến

- Khi ta lưu một giá trị xuống bộ nhớ ram thông qua biến . Làm sao để biết vị trí biến đó trong ram (ô nhớ nào) vì bản chất bộ nhớ là một dãy cá ô nhớ bắt đầu từ 0. Để biết biến đó lưu ở đâu ta dùng toán tử **&** . Việc chúng ta làm việc trên địa chỉ tương đương với việc chúng ta đang điều khiển trực tiếp bộ nhớ. Trong C cung cấp một dữ liệu chuyên lưu địa chỉ đó là con trỏ , với kiểu dữ liệu nào (int , char ...) sẽ có con trỏ tương ứng để lưu. cú pháp : kiểu_dữ_liệu* tên_con_trỏ = địa chỉ

Ví dụ: float nhiet_do = 7.5; float* p_nhiet_do = &nhiet_co;

Khi có con trỏ rồi , giống như ta chiếm được quyền điều khiển bộ nhớ . Ta chỉ cần cài lại giá trị vùng nhớ đó giá trị ta mong muốn qua cấu trúc cú pháp : *tên_con_trỏ = giá trị

*p_nhiet_do =20

Kết quả nhiet_do = 20

ví dụ : ta dùng cú pháp : float nhiet_do = 7.5 . ta đã yêu cầu hệ điều hành cấp một vùng nhớ trên ram để lưu giá trị 7.5, như vậy vị trí đó sẽ ngẫu nhiên. Nhưng ta lại muốn yêu cầu khác hãy lưu giá trị 7.5 vào vị trí thứ 10 trong ram , lúc này ta phải dùng kiến thức địa chỉ để lưu

- chọn một con trỏ để lưu địa chỉ đó (địa chỉ thứ 10): `float* p_nhiet_do = (float*)10`
- lưu giá trị xuống vùng nhớ thông qua con trỏ : `*p_nhiet_do = 7.5`